Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК Информатика и управление

КАФЕДРА ИУК3 Системы автоматического управления

ОТЧЕТ
по НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ

НА ТЕМУ:

Исследование подходов к построению веб-системы моделирования САУ

Студент группы ИУК3-71Б		Исаев А.А.
	(Подпись)	(И.О. Фамилия)
Руководитель НИР		Макаренков А.М.
	(Подпись)	(И.О. Фамилия)
Оценка руководителя НИР		
	Баллов, 30-50	(Дата)
Оценка за защиту		
	Баллов, 30-50	(Дата)
Итоговая оценка		
	Баллов	(по пятибалльной шкале)

Комиссия: _____ _____

_____ _____

_____ _____

Калуга, 2026

Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ
Заведующий кафедрой ИУК3
_____ (Д.В. Мельников)
«___» _____ 2026 г.

З А Д А Н И Е

на научно-исследовательскую работу
на тему:

Исследование подходов к построению веб-системы моделирования САУ

Задание:

1. Проанализировать существующие средства моделирования систем автоматического управления и сформулировать требования к веб-системе моделирования.
2. Исследовать варианты архитектуры веб-системы и выбрать, где и как выполнять расчёты.
3. Сравнить подходы к получению математической модели по структурной схеме и обосновать выбор.
4. Экспериментально сравнить численные методы решения уравнений модели: точность, устойчивость, затраты времени.
5. Определить состав видов анализа системы и обобщить результаты исследования.

Дата выдачи задания «___» _____ 2026 г.

Руководитель _____ Макаренков А.М.
(подпись, дата) (И.О. Фамилия)

Студент _____ Исаев А.А.
(подпись, дата) (И.О. Фамилия)

Содержание

Перечень сокращений и обозначений....4
Введение....5
1. Сбор и анализ материала, анализ литературы....7
1.1. Актуальность темы исследования....7
1.2. Обзор существующих средств моделирования....7
1.3. Требования к разрабатываемой системе....8
1.4. Объект, предмет, цель и задачи исследования....9
2. Проведение научного исследования, расчётов....10
2.1. Где выполнять расчёты: выбор архитектуры....10
2.2. Подходы к получению модели по схеме....11
2.3. Численные методы решения уравнений модели....13
2.4. Эксперимент 1: точность в зависимости от шага....13
2.5. Эксперимент 2: число шагов для заданной точности....15
2.6. Эксперимент 3: устойчивость метода Эйлера....15
2.7. Эксперимент 4: время расчёта....16
2.8. Виды анализа в системе....17
3. Обобщение полученных результатов....18
3.1. Научные и практические результаты....18
3.2. Ограничения и направления дальнейшей работы....18
Заключение....19
Список использованных источников....20
Приложение А. Листинги программы....21

Перечень сокращений и обозначений

САУ – система автоматического управления

ПФ – передаточная функция

A, B, C, D – матрицы модели $\dot{x} = Ax + Bu,\ y = Cx + Du$

RK4 – метод Рунге–Кутты 4-го порядка с постоянным шагом

RK45 – метод Дорманда–Принса с автоматическим выбором шага

h – шаг интегрирования

API – программный интерфейс сервера

JSON – текстовый формат обмена данными между браузером и сервером

Введение

Научно-исследовательская работа выполнялась по направлению подготовки 27.03.04 «Управление в технических системах». Она является исследовательской частью выпускной квалификационной работы «Веб-ориентированная система визуального моделирования систем автоматического управления».

Для изучения и проектирования систем автоматического управления (САУ) инженеры и студенты используют программы блочного моделирования. В них схема системы собирается из блоков-звеньев, а программа рассчитывает, как система ведёт себя во времени, устойчива ли она и какие у неё частотные характеристики. Самая известная такая программа, MATLAB/Simulink, для российских вузов стала труднодоступна, а другие программы требуют установки на каждый компьютер и не всегда показывают, как получен результат.

Поэтому разрабатывается веб-система моделирования: она открывается в обычном браузере и не требует установки. Прежде чем её строить, нужно ответить на несколько вопросов. Какие возможности есть у существующих программ и что из них нужно? Где выполнять расчёты — в браузере или на сервере? Как превращать нарисованную схему в уравнения? Каким численным методом решать эти уравнения? Этим вопросам и посвящена работа.

Цель работы — исследовать и обосновать подходы к построению веб-системы моделирования САУ: архитектуру, способ получения модели по схеме и численные методы.

Для этого решались задачи:

– проанализировать существующие средства моделирования и сформулировать требования к системе;

– сравнить варианты архитектуры и выбрать, где выполнять расчёты;

– сравнить подходы к получению модели по структурной схеме;

– экспериментально сравнить численные методы решения уравнений модели;

– определить состав видов анализа и обобщить результаты.

1. Сбор и анализ материала, анализ литературы

1.1. Актуальность темы исследования

Структурная схема — основной язык теории автоматического управления [2, 3]. Человеку удобно рассуждать звеньями и связями, а компьютеру — решать систему уравнений. Программа блочного моделирования соединяет одно с другим: пользователь рисует схему, программа считает.

С 2022 г. компания MathWorks, разработчик MATLAB/Simulink, прекратила работу в России, и получить новые лицензии для вузов стало практически невозможно. Отечественные и свободные программы есть, но большинство из них нужно устанавливать на компьютер, а коммерческие требуют лицензий. Для учебного процесса полезна среда, которая работает в браузере, бесплатна и показывает, какие уравнения она составила по схеме. Поэтому исследование подходов к построению такой среды актуально.

1.2. Обзор существующих средств моделирования

Рассмотрены пять наиболее известных средств блочного моделирования (таблица 1). Сведения взяты с сайтов разработчиков и из документации [5–9].

Таблица 1 – Средства моделирования САУ

Средство	**Как устроено**	**Доступ**	**Недостатки для учебных задач**
MATLAB / Simulink (MathWorks, США)	Блочные схемы, поблочный расчёт; общую модель схемы даёт отдельный пакет	Коммерческая лицензия, установка	С 2022 г. новые лицензии в РФ недоступны
Scilab / Xcos (Франция)	Блочные схемы гибридных систем	Бесплатно, открытый код, установка	Устаревший интерфейс, модель схемы пользователю не показывается
OpenModelica (консорциум OSMC)	Модели в виде уравнений на языке Modelica	Бесплатно, открытый код, установка	Сложна для начинающих: нужно изучать язык Modelica
SimInTech (Россия)	Блочные схемы, расчёт в реальном времени	Коммерческая лицензия, установка под Windows	Закрытый код, без лицензии возможности ограничены
Engee (Россия)	Блочные схемы и вычислительная среда	Через сайт, есть бесплатный режим	Закрытый код, требует регистрации и постоянного подключения

Из обзора видно, что ни одно средство не сочетает сразу три свойства, важные для учебного процесса: работу в браузере без установки, бесплатность и открытость — возможность увидеть, какие уравнения программа составила по схеме и как она их решает.

1.3. Требования к разрабатываемой системе

По результатам обзора сформулированы требования к веб-системе:

– работа в браузере, без установки программ на компьютер пользователя;

– редактор структурных схем с типовыми звеньями и подсистемами (блоками, внутри которых своя схема);

– расчёт переходных процессов с выбором численного метода;

– анализ устойчивости и частотных характеристик;

– показ полученной модели системы и объяснение, почему схему нельзя рассчитать, если это так;

– сохранение проектов и возможность повторить любой расчёт из консоли для исследований.

1.4. Объект, предмет, цель и задачи исследования

Объект исследования — веб-система визуального моделирования линейных непрерывных САУ.

Предмет исследования — подходы к её построению: архитектура, способ получения математической модели по схеме и численные методы решения уравнений модели.

Цель и задачи исследования сформулированы во введении.

2. Проведение научного исследования, расчётов

2.1. Где выполнять расчёты: выбор архитектуры

Веб-система состоит из двух частей: того, что работает в браузере пользователя (клиент), и того, что работает на сервере. Редактор схем и графики, очевидно, должны быть в браузере. Главный вопрос — где выполнять расчёты. Два варианта сравниваются в таблице 2.

Таблица 2 – Варианты размещения расчётов

Критерий	**Расчёт в браузере (JavaScript)**	**Расчёт на сервере (Python)**
Библиотеки линейной алгебры и решатели	Немногочисленны, менее проверены	NumPy и SciPy — стандарт научных расчётов [10, 11]
Нагрузка на компьютер пользователя	Вся на нём	Минимальная
Работа без сети	Возможна	Нужен доступ к серверу
Повторение расчёта для исследования	Только через интерфейс	Те же функции вызываются из консольных сценариев
Проверка правильности	Нужно дублировать расчёты	Один вычислительный модуль для всего

Выбран расчёт на сервере. Главная причина — точность и проверяемость: библиотеки NumPy и SciPy много лет используются в научных расчётах, а одни и те же функции вызываются и из интерфейса, и из консольных сценариев, которыми получены все таблицы этого отчёта. Недостаток — нужен доступ к серверу — для учебного класса несущественен.

Получившаяся архитектура показана на рисунке 1. Браузер отправляет схему на сервер в формате JSON — это текстовая запись списка блоков и соединений. Сервер (FastAPI) передаёт её вычислительному ядру, а результат возвращает браузеру. Проекты пользователей хранятся в базе данных SQLite.



Рисунок 1 – Архитектура веб-системы моделирования

Так выглядит окно системы в браузере (рисунок 2).

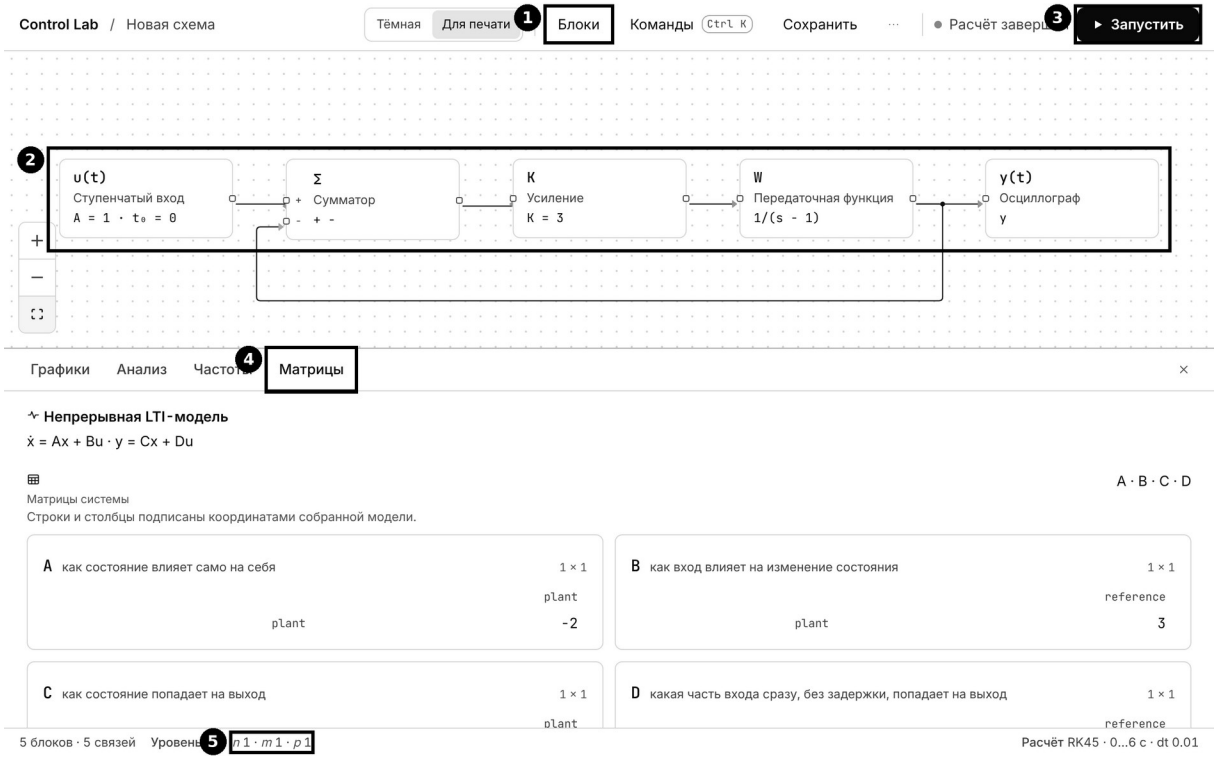


Рисунок 2 – Окно системы: (1) библиотека блоков, (2) схема, (3) запуск расчёта, (4) вкладка с моделью системы, (5) размеры модели

2.2. Подходы к получению модели по схеме

Чтобы рассчитать схему, программа должна превратить её в уравнения. Рассматривались три подхода (рисунок 3 и таблица 3).

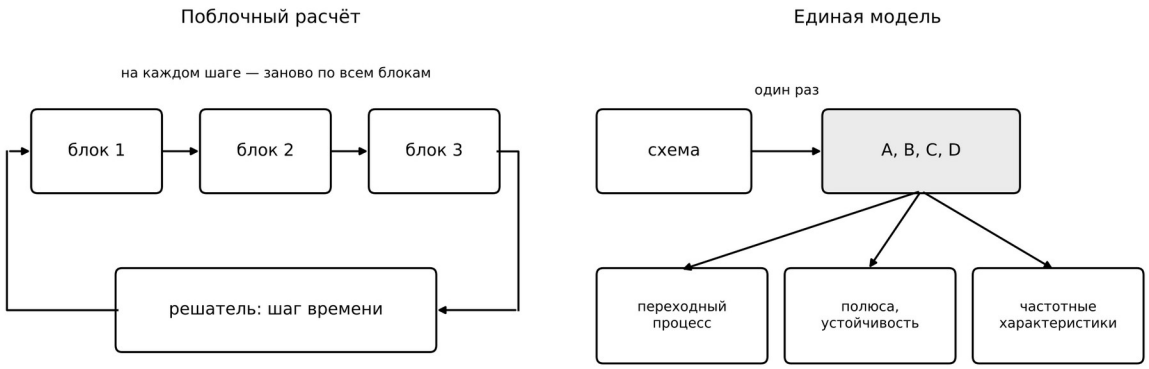


Рисунок 3 – Поблочный расчёт и единая модель

Таблица 3 – Подходы к получению модели

Подход	**Суть**	**Плюсы**	**Минусы**
Поблочный расчёт	На каждом шаге времени блоки по очереди считают свои выходы и производные — так работает обычное моделирование в Simulink	Прост, подходит для нелинейных блоков	Нет общих уравнений системы; не считает петли, где сигнал проходит мгновенно
Уравнения на языке моделирования	Пользователь описывает систему уравнениями, программа упрощает их сама (OpenModelica)	Очень гибок	Избыточен для структурных схем САУ, сложен для студентов
Единая модель в пространстве состояний	Схема один раз превращается в матрицы A, B, C, D всей системы	Сразу даёт полюса, ПФ, частотные характеристики; модель можно показать	Только линейные звенья

Первая версия ядра системы использовала поблочный расчёт. На практике выяснилось, что он не справляется со схемами, в которых сигнал проходит по петле мгновенно (через усилители и звенья с прямой передачей), и не даёт матриц всей системы. Единая модель такие петли решает как систему линейных уравнений. Её правильность проверена в курсовом проекте сравнением с независимым расчётом в MATLAB/Octave на контуре с ПИ-регулятором, схеме с перекрёстными обратными связями и электрогидравлическом следящем приводе 8-го порядка: передаточные функции и переходные процессы совпали с точностью вычислений.

Кроме того, только единая модель даёт то, что нужно для анализа САУ: полюса системы — это собственные числа матрицы A, передаточная функция — $C(sI - A)^{-1}B + D$, а точное решение для проверки численных методов (раздел

2.3) тоже получается из матриц. Ограничение — только линейные звенья — для курса теории линейных систем несущественно.

Поэтому выбран подход единой модели. Алгоритм построения матриц по схеме и его проверка подробно разработаны в курсовом проекте «Разработка подсистемы построения математической модели САУ в пространстве состояний по структурной схеме».

2.3. Численные методы решения уравнений модели

Модель $\dot{x} = A\,x + B\,r$ — это система дифференциальных уравнений. Решать её приходится численно: время делится на шаги длиной h, и состояние пересчитывается от шага к шагу. Сравнивались три метода [4]:

– метод Эйлера — на каждом шаге один раз вычисляет скорость изменения состояния и делает шаг по ней. Он самый простой, но неточный: скорость в начале шага считается постоянной на весь шаг;

– метод Рунге–Кутты 4-го порядка (RK4) — вычисляет скорость четыре раза внутри шага и усредняет, поэтому намного точнее при том же шаге;

– метод RK45 (Дорманда–Принса) — сам выбирает шаг так, чтобы ошибка не превышала заданную.

Эталоном служило точное решение. Для линейной модели его можно вычислить через матричную экспоненту — это ещё одно преимущество единой модели. Исследования выполнены сценарием:

```
cd backend
python scripts/nir_solver_study.py
```

2.4. Эксперимент 1: точность в зависимости от шага

Порядок метода показывает, как быстро уменьшается ошибка при уменьшении шага. Он вычислялся как $p = \log_2 \dfrac{\varepsilon(h)}{\varepsilon(h/2)}$, где $\varepsilon(h)$ — наибольшая ошибка при шаге h. Результаты для контура с ПИД-регулятором приведены в таблице 4 и на рисунке 4.

Таблица 4 – Ошибка методов на контуре с ПИД-регулятором

h, с	Эйлер	RK4	RK45	p Эйлер	p RK4
0,2	$1{,}2\cdot10^{2}$	$3{,}9\cdot10^{-3}$	$7{,}3\cdot10^{-9}$	—	—
0,1	$2{,}1\cdot10^{0}$	$2{,}4\cdot10^{-4}$	$7{,}3\cdot10^{-9}$	5,83	4,04
0,05	$2{,}8\cdot10^{-1}$	$1{,}5\cdot10^{-5}$	$7{,}3\cdot10^{-9}$	2,94	4,02
0,025	$1{,}1\cdot10^{-1}$	$9{,}1\cdot10^{-7}$	$7{,}4\cdot10^{-9}$	1,38	4,01
0,0125	$4{,}7\cdot10^{-2}$	$5{,}7\cdot10^{-8}$	$7{,}4\cdot10^{-9}$	1,15	4,01

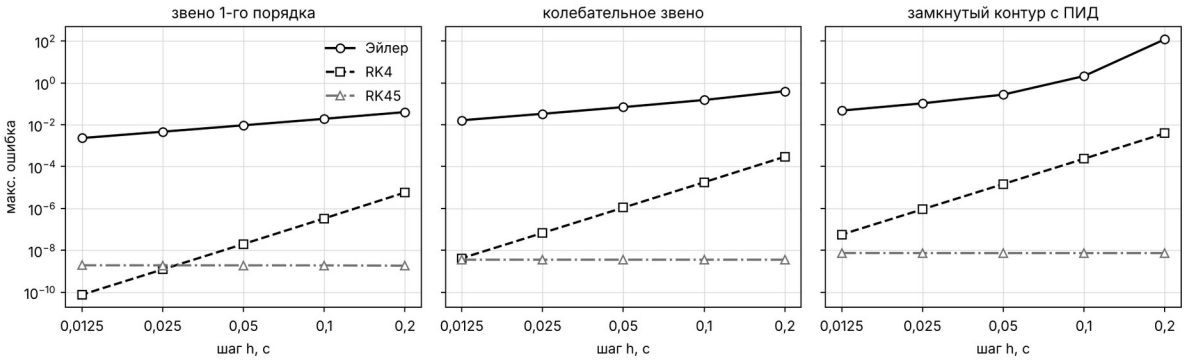


Рисунок 4 – Ошибка численных методов в зависимости от шага

Порядок метода Эйлера приближается к 1, метода RK4 — к 4, как и предсказывает теория [4]. На практике это значит: если уменьшить шаг вдвое, ошибка метода Эйлера уменьшится вдвое, а ошибка RK4 — в 16 раз. Ошибка RK45 почти не зависит от шага вывода, потому что метод сам выбирает свой внутренний шаг. Разницу хорошо видно и на графике переходного процесса при одном и том же шаге $h=0{,}1$ (рисунок 5): кривая RK4 совпадает с точным решением, а кривая метода Эйлера заметно отклоняется.

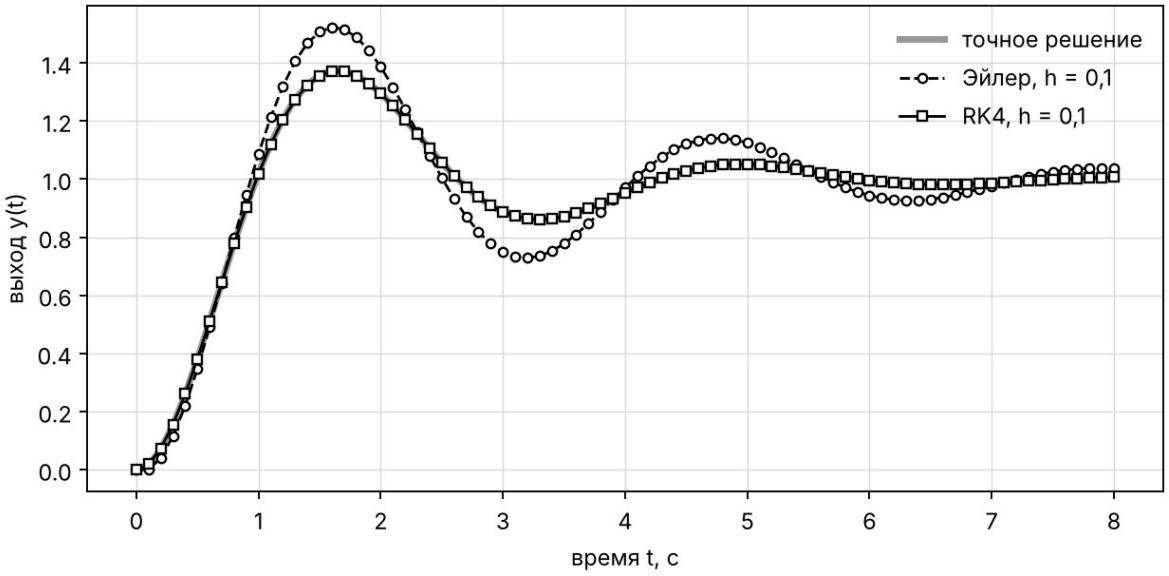


Рисунок 5 – Переходный процесс колебательного звена: Эйлер и RK4 при h = 0,1

2.5. Эксперимент 2: число шагов для заданной точности

Для каждого метода подбирался наибольший шаг, при котором ошибка на интервале 8 с не превышает 0,001 (таблица 5).

Таблица 5 – Шаг и число шагов для ошибки не больше 0,001

Схема	Шаг Эйлер	Шагов Эйлер	Шаг RK4	Шагов RK4	Выигрыш RK4
звено 1-го порядка	0,00390625	2048	0,5	16	128 раз
колебательное звено	0,000488281	16384	0,25	32	512 раз
замкнутый контур с ПИД	0,000244141	32768	0,125	64	512 раз

Методу Эйлера нужно в 128–512 раз больше шагов, чем RK4. Даже с учётом того, что шаг RK4 в четыре раза дороже, RK4 выигрывает в десятки раз.

2.6. Эксперимент 3: устойчивость метода Эйлера

У численного метода есть ещё одно свойство — устойчивость: при слишком большом шаге расчёт может «взорваться», хотя сама система устойчива. Для звена $\frac{1}{T s+1}$ один шаг метода Эйлера умножает отклонение от

установившегося значения на $(1-h/T)$. Если $h>2T$, модуль этого множителя больше единицы и ошибка растёт с каждым шагом (таблица 6, рисунок 6).

Таблица 6 – Звено 1-го порядка (T = 1 с): отклонение при t = 30 с

h / T	Множитель за шаг	Эйлер	Итог Эйлер	RK4	Итог RK4
0,5	0,5	0	сходится	$9{,}6\cdot10^{-14}$	сходится
1,0	0,0	0	сходится	$1{,}7\cdot10^{-13}$	сходится
1,5	0,5	$9{,}5\cdot10^{-7}$	сходится	$5{,}5\cdot10^{-12}$	сходится
1,9	0,9	$1{,}9\cdot10^{-1}$	сходится	$5{,}6\cdot10^{-9}$	сходится
2,1	1,1	$3{,}8\cdot10^{0}$	расходится	$9{,}7\cdot10^{-7}$	сходится
2,5	1,5	$1{,}3\cdot10^{2}$	расходится	$5{,}5\cdot10^{-3}$	сходится

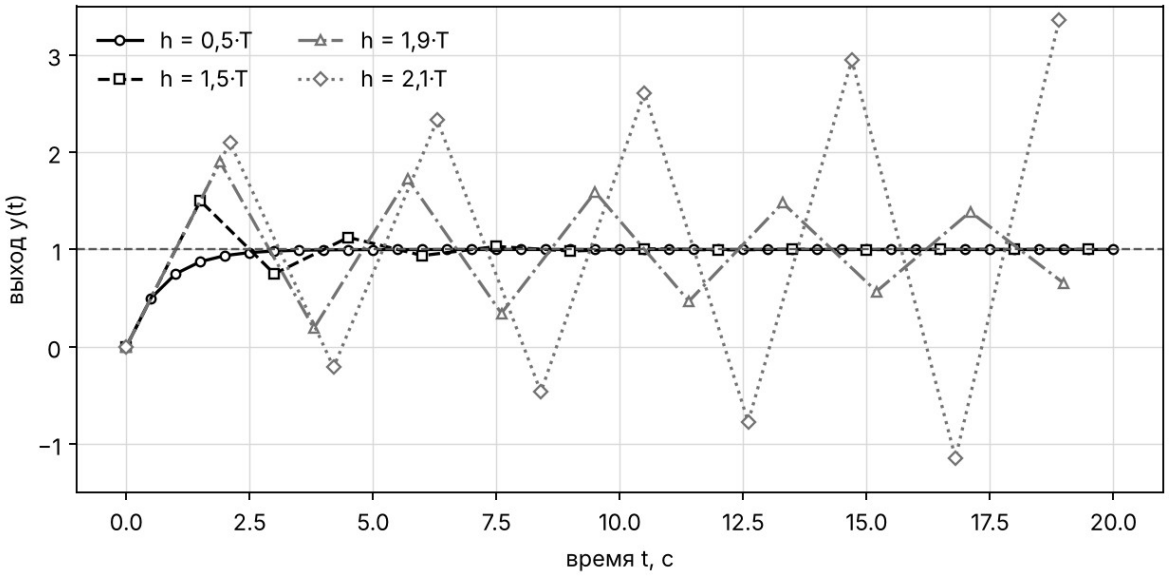


Рисунок 6 – Метод Эйлера при разных шагах

Метод RK4 устойчив при заметно больших шагах, но и у него есть граница. Поэтому, если пользователь выбрал RK4 с постоянным шагом, система до расчёта проверяет шаг по полюсам модели (листинг 4 приложения А). Если численное решение будет неустойчивым, расчёт не запускается, а пользователь получает сообщение с допустимым шагом. Это тоже возможно только благодаря единой модели: полюса известны заранее.

2.7. Эксперимент 4: время расчёта

Время расчёта контура с ПИД-регулятором на интервале 10 с приведено в таблице 7.

Таблица 7 – Время расчёта (контур с ПИД, 10 с, h = 0,01)

Метод	Вычислений на шаг	Время, мс	Ошибка
Эйлер	1	10,13	$3{,}7\cdot10^{-2}$
RK4	4	27,68	$2{,}3\cdot10^{-8}$
RK45	переменно	31,78	$7{,}4\cdot10^{-9}$

Все методы рассчитывают учебную схему за десятки миллисекунд, то есть пользователь получает результат практически сразу. При этом ошибка RK4 и RK45 в миллионы раз меньше, чем у метода Эйлера. Поэтому в системе по умолчанию используется RK45, RK4 доступен для расчёта с постоянным шагом, а метод Эйлера оставлен только в исследовательских сценариях для сравнения.

2.8. Виды анализа в системе

Единая модель позволяет получить основные характеристики САУ (таблица 8). Все они вычисляются по одним и тем же матрицам, поэтому согласованы между собой.

Таблица 8 – Виды анализа и как они получаются из модели

Вид анализа	Как получается
Переходный процесс	численное решение уравнений модели (разделы 2.3–2.7)
Показатели качества: перерегулирование, время регулирования	по переходному процессу и установившемуся значению $\left(D - C\,A^{-1}B\right)r$
Полюса и устойчивость	собственные числа матрицы A
Частотные характеристики	$W(j\omega)=C(j\omega I - A)^{-1}B+D$
Модель системы	матрицы A, B, C, D и то, какому блоку принадлежит каждая переменная

3. Обобщение полученных результатов

3.1. Научные и практические результаты

В ходе НИР получены следующие результаты.

– Проанализированы пять средств моделирования САУ; показано, что ни одно не сочетает работу в браузере, бесплатность и открытость, и сформулированы требования к веб-системе.

– Обоснована архитектура: редактор и графики в браузере, расчёты на сервере на Python с NumPy и SciPy, одно вычислительное ядро для интерфейса и для исследований.

– Обоснован подход единой модели в пространстве состояний: в отличие от поблочного расчёта он решает петли без инерционных звеньев и сразу даёт полюса и передаточную функцию системы.

– Экспериментально подтверждены порядки точности методов Эйлера (1) и RK4 (4); методу Эйлера для той же точности нужно в 128–512 раз больше шагов.

– Показано, что метод Эйлера теряет устойчивость при $h>2T$, и обоснована проверка шага по полюсам модели до расчёта.

Практический результат — архитектура и вычислительное ядро веб-системы визуального моделирования, на которых строится выпускная квалификационная работа.

3.2. Ограничения и направления дальнейшей работы

– система рассчитана на линейные звенья; для нелинейных элементов (насыщение, реле) потребуется сочетать единую модель с поблочным расчётом нелинейной части;

– входные воздействия пока только ступенчатые;

– алгоритм построения единой модели по схеме разрабатывается в курсовом проекте.

Заключение

В результате НИР исследованы подходы к построению веб-системы моделирования систем автоматического управления.

Обзор существующих средств показал, что для учебного процесса нужна среда, которая работает в браузере, бесплатна и показывает полученную модель. Для неё выбрана клиент-серверная архитектура с расчётами на сервере, что обеспечивает точность, проверяемость и возможность повторять расчёты из консоли.

Сравнение подходов к получению модели показало преимущество единой модели в пространстве состояний: она рассчитывает все тестовые схемы, включая недоступные поблочному расчёту, и сразу даёт полюса, передаточные функции и частотные характеристики.

Эксперименты с численными методами подтвердили теоретические порядки точности и показали, что методы Рунге–Кутты в сотни раз экономнее метода Эйлера при той же точности; для метода Эйлера найдена граница устойчивости по шагу.

Результаты использованы при разработке веб-системы визуального моделирования — выпускной квалификационной работы.

Список использованных источников

1. Финошин А.В., Корнюшин Ю.П., Краснощеченко В.И. Научно-исследовательская работа: практикум. – Калуга: КФ МГТУ им. Н.Э. Баумана, 2024. – 17 с.

2. Бесекерский В.А., Попов Е.П. Теория систем автоматического регулирования. – М.: Директмедиа, 2021. – 747 с.

3. Ким Д.П. Теория автоматического управления. Т. 1. Линейные системы. – М.: Физматлит, 2016. – 312 с.

4. Hairer E., Nørsett S.P., Wanner G. Solving Ordinary Differential Equations I: Nonstiff Problems. – Berlin; Heidelberg: Springer, 2008. – 528 p.

5. Simulink – Simulation and Model-Based Design [Электронный ресурс]. – URL: https://www.mathworks.com/products/simulink.html (дата обращения: 20.10.2026).

6. Scilab: Xcos [Электронный ресурс]. – URL: https://www.scilab.org/software/xcos (дата обращения: 20.10.2026).

7. OpenModelica [Электронный ресурс]. – URL: https://openmodelica.org/ (дата обращения: 20.10.2026).

8. SimInTech – среда динамического моделирования технических систем [Электронный ресурс]. – URL: https://simintech.ru/ (дата обращения: 20.10.2026).

9. Engee – среда модельно-ориентированного проектирования [Электронный ресурс]. – URL: https://start.engee.com/ (дата обращения: 20.10.2026).

10. NumPy Documentation [Электронный ресурс]. – URL: https://numpy.org/doc/ (дата обращения: 20.10.2026).

11. SciPy API Reference [Электронный ресурс]. – URL: https://docs.scipy.org/doc/scipy/reference/ (дата обращения: 20.10.2026).

Приложение А. Листинги программы

Листинг 1 – Метод Эйлера (app/simulation/solvers.py)

```
def euler_integrate(
    rhs: Rhs,
    x0: np.ndarray,
    t_grid: np.ndarray,
    *,
    breakpoints: Sequence[float] = (),
) -> np.ndarray:
    """Explicit Euler on the given grid: x_{k+1} = x_k + h·f(t_k, x_k).

    First-order reference method for the numerical study; it is not offered by
    the API. The derivative is taken at the left end of each step, so a jump at
    a breakpoint grid node takes effect from the step that starts there.
    """

    _validate_time_grid(t_grid)
    del breakpoints  # the left-point rule never evaluates past the step start
    if x0.size == 0:
        return np.zeros((0, t_grid.size))

    trajectory = np.zeros((x0.size, t_grid.size))
    trajectory[:, 0] = x0
    x = x0.astype(float).copy()
    for index in range(t_grid.size - 1):
        left, right = float(t_grid[index]), float(t_grid[index + 1])
        with np.errstate(over="ignore", invalid="ignore"):  # divergence is reported
below
            x = x + (right - left) * rhs(left, x)
        if not np.all(np.isfinite(x)):
            raise SolverError(f"Эйлер: состояние перестало быть конечным в момент
t={right:g}.")
        trajectory[:, index + 1] = x
    return trajectory
```

Листинг 2 – Метод Рунге–Кутты 4-го порядка (app/simulation/solvers.py)

```
def rk4_integrate(
    rhs: Rhs,
    x0: np.ndarray,
    t_grid: np.ndarray,
    *,
    breakpoints: Sequence[float] = (),
) -> np.ndarray:
    """Classical RK4 on the given grid; every breakpoint must be a grid node.

    At a step that ends exactly on a breakpoint the last stage is evaluated
    just before it (left limit of the input), so the jump is applied only at
    the start of the next step.
    """

    _validate_time_grid(t_grid)
    if x0.size == 0:
        return np.zeros((0, t_grid.size))

    ends_on_jump = np.isin(t_grid[1:], np.asarray(breakpoints, dtype=float))
    trajectory = np.zeros((x0.size, t_grid.size))
    trajectory[:, 0] = x0
    x = x0.astype(float).copy()
    for index in range(t_grid.size - 1):
        left, right = float(t_grid[index]), float(t_grid[index + 1])
        h = right - left
        k4_time = float(np.nextafter(right, left)) if ends_on_jump[index] else right
        k1 = rhs(left, x)
        k2 = rhs(left + 0.5 * h, x + 0.5 * h * k1)
```

```
        k3 = rhs(left + 0.5 * h, x + 0.5 * h * k2)
        k4 = rhs(k4_time, x + h * k3)
        x = x + (h / 6.0) * (k1 + 2.0 * k2 + 2.0 * k3 + k4)
        if not np.all(np.isfinite(x)):
            raise SolverError(f"RK4: состояние перестало быть конечным в момент
t={right:g}.")
        trajectory[:, index + 1] = x
    return trajectory
```

Листинг 3 – Точное решение через матричную экспоненту (app/simulation/solvers.py)

```
def exact_lti_integrate(
    a: np.ndarray,
    b: np.ndarray,
    source_values: Callable[[float], np.ndarray],
    x0: np.ndarray,
    t_grid: np.ndarray,
) -> np.ndarray:
    """Exact solution of x' = A x + B r for r constant on every grid interval.

    x(t + h) = e^{Ah} x(t) + G(h) B r,  G(h) = integral_0^h e^{A s} ds,
    both taken from the matrix exponential of [[A, I], [0, 0]] h. The grid
    must contain every input jump. Used as the reference for RK4 and RK45.
    """

    _validate_time_grid(t_grid)
    n = x0.size
    if n == 0:
        return np.zeros((0, t_grid.size))

    augmented = np.zeros((2 * n, 2 * n))
    augmented[:n, :n] = a
    augmented[:n, n:] = np.eye(n)
    cache: dict[float, tuple[np.ndarray, np.ndarray]] = {}

    trajectory = np.zeros((n, t_grid.size))
    trajectory[:, 0] = x0
    x = x0.astype(float).copy()
    for index in range(t_grid.size - 1):
        h = float(t_grid[index + 1] - t_grid[index])
        # Steps of a uniform grid differ only by rounding; 13 significant
        # digits keep one propagator per nominal step.
        key = float(f"{h:.12e}")
        if key not in cache:
            exponential = expm(augmented * key)
            cache[key] = (exponential[:n, :n], exponential[:n, n:])
        phi, gamma = cache[key]
        x = phi @ x + gamma @ (b @ source_values(float(t_grid[index])))
        trajectory[:, index + 1] = x
    return trajectory
```

Листинг 4 – Проверка шага RK4 по полюсам модели (app/analysis/stability.py)

```
def rk4_step_check(a: np.ndarray, step: float) -> dict[str, Any]:
    """Compare the RK4 amplification with the exact one, e^(h lambda), per mode.

    A mode that is stable or marginal physically (Re lambda <= tol) must not be
    amplified by the method: |R(h lambda)| > 1 is numerical instability and
    the step is rejected. For a physically growing mode the growth is real;
    only a large relative error of the per-step factor is reported.
    """

    if a.size == 0:
        return {"stable": True, "unstable_modes": [], "max_step": None,
"inaccurate_growing_modes": []}
```

```
    poles = np.linalg.eigvals(a)
    unstable: list[complex] = []
    limits: list[float] = []
    inaccurate: list[complex] = []
    for pole in poles:
        z = step * pole
        amplification = abs(rk4_amplification(z))
        # Relative to |lambda|: a fast block elsewhere must not turn a slowly
        # growing mode into an "axis" mode.
        if pole.real <= AXIS_RTOL * abs(pole):
            if amplification > 1.0 + 1e-12:
                unstable.append(complex(pole))
                limits.append(_rk4_step_limit(complex(pole), step))
        else:
            exact = abs(np.exp(z))
            if abs(amplification - exact) > 0.01 * exact:
                inaccurate.append(complex(pole))
    return {
        "stable": not unstable,
        "unstable_modes": unstable,
        "max_step": min(limits) if limits else None,
        "inaccurate_growing_modes": inaccurate,
    }
```

Листинг 5 – Эксперимент «шаг для заданной точности»

(scripts/nir_solver_study.py)

```
def step_for_tolerance(tolerance: float = 1e-3) -> list[dict]:
    """Наибольший шаг из ряда h = 0.5 / 2^k, при котором ошибка не превышает
tolerance."""

    rows = []
    for case, diagram in CASES.items():
        model = model_of(diagram)
        row: dict = {"схема": case}
        for name in ("Эйлер", "RK4"):
            dt = 0.5
            while dt > 1e-5:
                grid = grid_of(8.0, dt)
                try:
                    error = float(np.max(np.abs(simulate(model, METHODS[name], grid) -
exact(model, grid))))
                except SolverError:
                    error = np.inf
                if error <= tolerance:
                    break
                dt /= 2
            row[f"шаг {name}"] = f"{dt:g}"
            row[f"шагов {name}"] = int(round(8.0 / dt))
        row["выигрыш RK4, раз"] = f"{row['шагов Эйлер'] / row['шагов RK4']:.0f}"
        rows.append(row)
    return rows
```